

EL403 – Assignment 4

Karthik Maddimsetti (202304006) and Het Patel (202304033), *Dhirubhai Ambani University, Gandhinagar.*

About the document — In this document, we have presented our implementation of an optimized design of FIR from the given information and for the specifications.

Given Information:

Filter Specification:

- Filter Type: Low-Pass Filter
- Cut-off frequency at 6 dB = 1kHz
- Stop-band attenuation of 20 dB = 1.1 kHz
- Number of Taps (N) = upto 15

Coefficients:

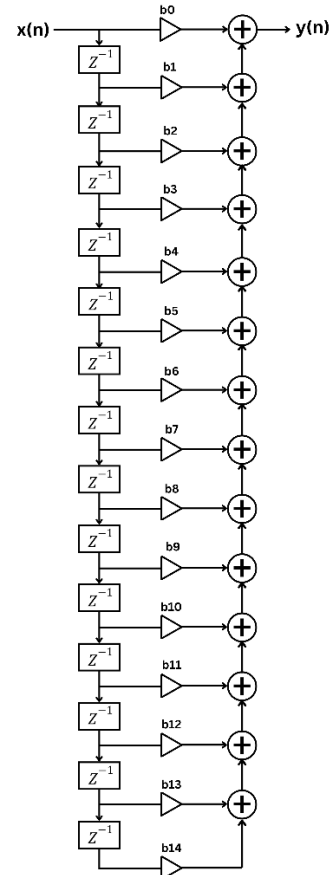
b = [-0.04557, 0, 0.06366, 0, -0.1061, 0, 0.3183, 0.5, 0.3183, 0, -0.1061, 0, 0.06366, 0, -0.04557]

Input samples and Coefficients are of 8-bit.

As discussed in the previous report,

- sampling frequency of the system is 4000 Hz,
- the samples and the coefficients are represented in Q1.7 format,
- quantized coefficients are
 $[-0.046875, 0, 0.0625, 0, -0.109375, 0, 0.3203125, 0.5, 0.3203125, 0, -0.109375, 0, 0.0625, 0, -0.046875]$

iv. For Direct Form realization (with Finite Word length):



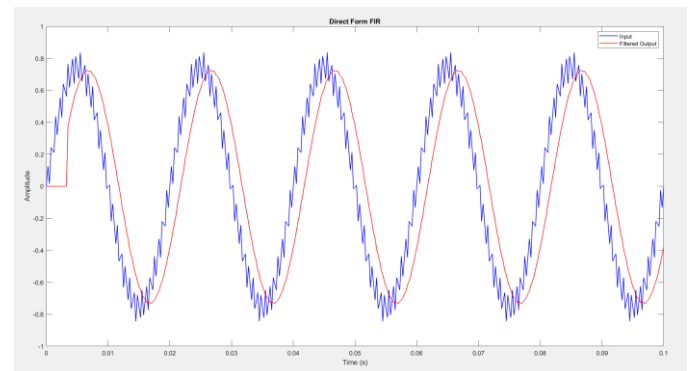
Resources needed:

- 15 Multipliers
- 14 Adders
- 14 Delay Elements
- 15 Memory Elements

Fig-1 : Direct Form Architecture.

Results from the MATLAB Model:

Input and Output Plot:



$$SQNR = 10 \log_{10} \left(\frac{P_{signal}}{P_{quantnoise}} \right) = 20 \log_{10} \left(\frac{yq}{y-yq} \right) = 26.03 \text{ dB}$$

Power Interpretation:

$$P_{\text{signal}} = 10^{2.603} P_{\text{quant noise}} \approx 401 P_{\text{quant noise}}$$

Implementation of the Optimized structure in Verilog – Structural.

As mentioned in the previous report, we have optimized the Direct Form structure to the following structure exploiting the following facts:

1. Alternate zeros in the coefficients makes the number of multipliers need half.
2. Positive Symmetry: The samples that are to be multiplied with the same coefficients can be grouped and pre-added before multiplying with the coefficients, again halving the multiplier requirement.

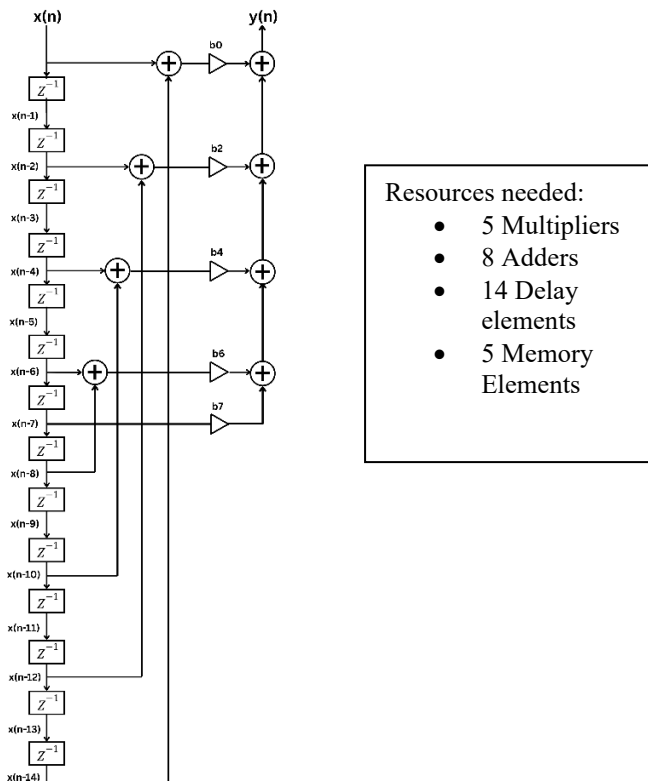


Fig-2. Optimized Architecture

Word-length decisions:

- 8-bit adders can be used for pre-additions and truncate the sum to 8-bit.
- 8-bit multipliers for multiplying the sums and truncate the product to 8-bit.
- 8-bit Multi-operand adders for adding the products and truncate the accumulator to 8-bit.

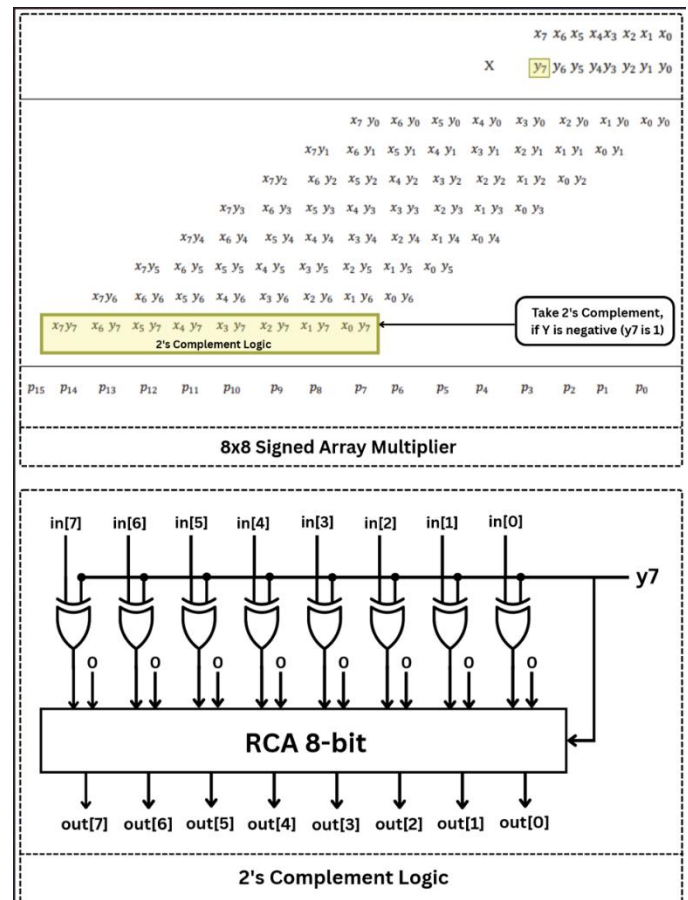
Choice of Adders and Multipliers:

- i. For pre-additions, we used *simple 8-bit RCAs* and truncated the result to 8-bits is fed into the multiplier.
- ii. For multipliers, we used a *modified the version of unsigned 8x8 Array Multiplier* (discussed below) to handle signed 2's complement multiplication and fed the products bits – 15th bit to 8th bit (dropping the redundant sign bit and 7 LSBs) into the final adder.
- iii. For final addition/subtraction, we have used a *modified version of 6-operand multi-operand CSA-Tree Adder* (discussed below).

These choices were made based on the fact that, the input new samples come into the filter every 250 μ s (4000 Hz) which is far higher than the worst-case propagation delay of the combinational logic (.....). Even if the hardware is made faster, the next stage will need to wait for the next sample, so we prioritized minimizing hardware requirements.

Structure of Multiplier:

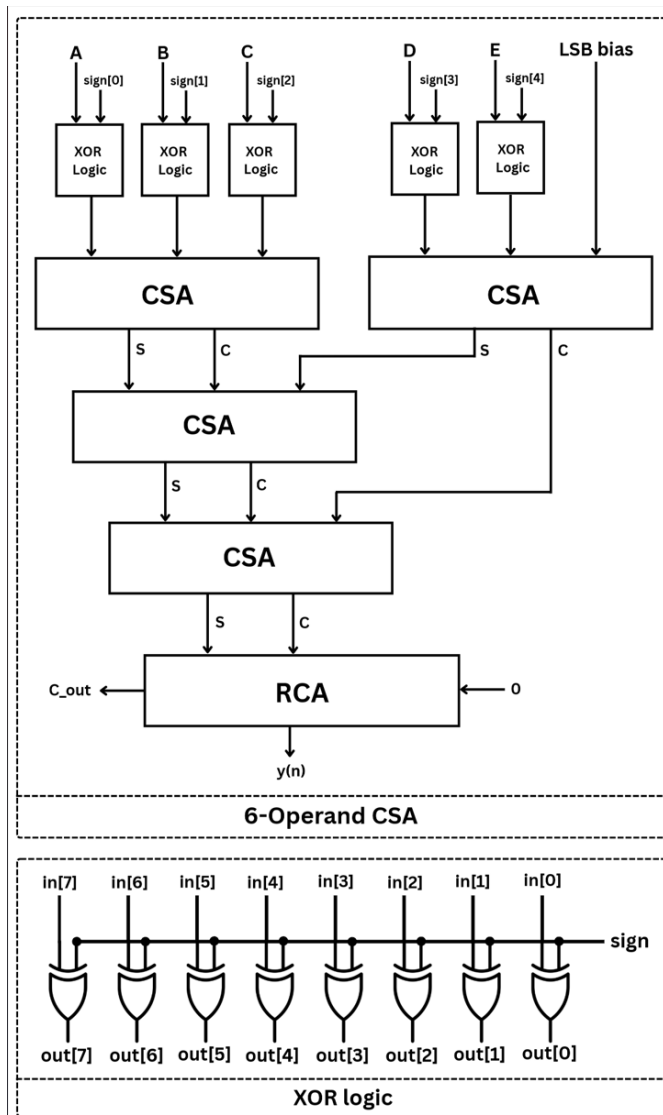
We have modified the 8×8 Array multiplier designed earlier to accommodate a unsigned $\times$ signed multiplication. We have assumed multiplicand to be positive (i.e., we will multiply the absolute value of the coefficient and add/subtract based on the sign of the coefficient). The below figures show the logical and structural modification in the multiplier designed previously.



Modification: Since only the MSB of the Multiplier (sample) has a negative weight, if the sample is negative, the partial products formed by multiplying the MSB of the sample should be taken in its 2's complement form otherwise, a string of zeroes. This is achieved by the (XORs + RCA) structure shown above.

Structure of CSA-Tree for Addition/Subtraction:

We have modified the CSA-Tree for Add/Subtract based on the sign of the coefficient as shown in the figure below.



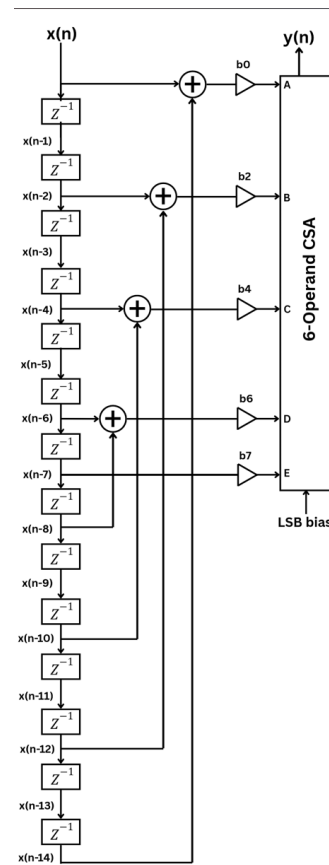
Sign[0] = b0, sign[1] = b2, sign[2] = b4, sign[3] = b6, sign[4] = b7 and sign of LSB bias is taken as always positive.

Modification:

If the sign of the coefficient is negative, a subtraction needs to be done since we have multiplied the sample with absolute value of the sample earlier. For subtraction we will need to add the 2's complement of the product. 2's complement is generated by using XOR logic shown above and add/subtraction decision taken using the sign bits of the coefficients. And LSB bias needs to be added to compute the 2's complement from the inverted bits.

The LSB bias will be equal to the number of the subtractions that are to be done. For eg, $A + B - C - D + E + \text{LSB bias}$ will require LSB bias to be $8'd2$.

The final structure is as shown below,



Resources needed:

- 5 8×8 (Unsigned × Signed) Array Multipliers
- 4 8-bit RCAs
- 3-stage (CSA-Tree) Multi-6 operand Adder/Subtractor + RCA
- 14 8-bit Registers
- 5 Memory Elements

Verilog HDL Code:

Top FIR Module:

```
module FIR(sample,clk,out);
input [7:0] sample;
input clk;
output [7:0] out;

wire [7:0] w_q [13:0];
wire c[3:0];
wire [7:0] pre_sum0, pre_sum1, pre_sum2, pre_sum3;
wire [15:0] p0,p1,p2,p3,p4;

//coefficients in Q1.7 format
reg [7:0] coeff0 = 8'b11111010; // - 0.046875
reg [7:0] coeff1 = 8'b00001000; // 0.0625
reg [7:0] coeff2 = 8'b11110010; // - 0.109375
reg [7:0] coeff3 = 8'b00101001; // 0.3203125
reg [7:0] coeff4 = 8'b01000000; // 0.5

wire [10:0] final_sum;
```

```

wire final_cout;
wire [5:0] sign;

assign sign =
{1'b0,coeff4[7],coeff3[7],coeff2[7],coef
f1[7],coeff0[7]};

d_reg r0 (.in(sample), .clk(clk),
.out(w_q[0]));
d_reg r1 (.in(w_q[0]), .clk(clk),
.out(w_q[1]));
d_reg r2 (.in(w_q[1]), .clk(clk),
.out(w_q[2]));
d_reg r3 (.in(w_q[2]), .clk(clk),
.out(w_q[3]));
d_reg r4 (.in(w_q[3]), .clk(clk),
.out(w_q[4]));
d_reg r5 (.in(w_q[4]), .clk(clk),
.out(w_q[5]));
d_reg r6 (.in(w_q[5]), .clk(clk),
.out(w_q[6]));
d_reg r7 (.in(w_q[6]), .clk(clk),
.out(w_q[7]));
d_reg r8 (.in(w_q[7]), .clk(clk),
.out(w_q[8]));
d_reg r9 (.in(w_q[8]), .clk(clk),
.out(w_q[9]));
d_reg r10(.in(w_q[9]), .clk(clk),
.out(w_q[10]));
d_reg r11(.in(w_q[10]), .clk(clk),
.out(w_q[11]));
d_reg r12(.in(w_q[11]), .clk(clk),
.out(w_q[12]));
d_reg r13(.in(w_q[12]), .clk(clk),
.out(w_q[13]));

RCA a0(.A(sample), .B(w_q[13]),
.CIN(1'b0), .COUT(c[0]),
.SUM(pre_sum0));
RCA a1(.A(w_q[1]), .B(w_q[11]),
.CIN(1'b0), .COUT(c[1]),
.SUM(pre_sum1));
RCA a2(.A(w_q[3]), .B(w_q[9]),
.CIN(1'b0), .COUT(c[2]),
.SUM(pre_sum2));
RCA a3(.A(w_q[5]), .B(w_q[7]),
.CIN(1'b0), .COUT(c[3]),
.SUM(pre_sum3));

// inputs to the multipliers will be
truncated (LSB dropped) to 8 bits
{cout,sum[7:1]}
Array_8x8_neg_xor m0(.A(coeff0),
.X({c[0],pre_sum0[7],pre_sum0[7:1]}),
.Pro(p0));
Array_8x8_neg_xor m1(.A(coeff1),
.X({c[1],pre_sum1[7],pre_sum1[7:1]}),
.Pro(p1));
Array_8x8_neg_xor m2(.A(coeff2),
.X({c[2],pre_sum2[7],pre_sum2[7:1]}),
.Pro(p2));
Array_8x8_neg_xor m3(.A(coeff3),
.X({c[3],pre_sum3[7],pre_sum3[7:1]}),
.Pro(p3));

```

```

Array_8x8_neg_xor m4(.A(coeff4),
.X(w_q[6]), .Pro(p4));

```

```

//inputs to the multiplier operand adder
will be truncated (LSBs dropped) to 8
bits

```

```

signed_CSA_6op_8bit
Add(.A(p0[14:7]),.B(p1[14:7]),.C(p2[14:7
]),.D(p3[14:7]),.E(p4[14:7]),.sign(sign)
,.lsb_bias(8'd2), .SUM(final_sum));

```

```

assign out = final_sum[7:0];

```

CSA + XOR logic Module

```

module
signed_CSA_6op_8bit(A,B,C,D,E,lsb_bias,s
ign,SUM,cout);
input[7:0] A,B,C,D,E,lsb_bias;
input[5:0] sign;
output [7:0] SUM;
output cout;
wire [7:0] S1,C1,S2,C2,S3,C3;
wire [8:0] S4,C4;
wire [9:0] sum1;
signed_CSA_3_op_8bit
a1(.A(A),.B(B),.C(C),.sign(sign[2:0]),.S
UM(S1),.COUT(C1));
signed_CSA_3_op_8bit
a2(.A(D),.B(E),.C(lsb_bias),.sign(sign[5
:3]),.SUM(S2),.COUT(C2));

```

```

CSA_3_op_8bit_2
a3(.A(S1),.B({C1[6:0],1'b0}),.C(S2),.SUM
(S3),.COUT(C3));
CSA_3_op_9bit_2
a4(.A({C1[7],S3}),.B({C3,1'b0}),.C({C2,1
'b0}),.SUM(S4),.COUT(C4));

```

```

RCA #(N(10))
R1(.A({1'b0,S4}),.B({C4,1'b0}),.CIN(1'b0
),.SUM(sum1),.COUT(cout));
assign SUM = sum1[7:0];

```

```

endmodule

```

Unsigned × Signed 8 × 8 Array Multiplier module:

```

///// A is +ve and B can be positive or
negative [ 8th bit is the sign bit ]

```

```

module Array_8x8_neg_xor(A,X,Pro);
input [7:0] A, X;
output [15:0] Pro;

```

```

    wire [7:0]
    ax0,ax1,ax2,ax3,ax4,ax5,ax6,ax7;

    partial_product_8x8
    PP(A,X,ax0,ax1,ax2,ax3,ax4,ax5,ax6,ax7);

    assign Pro[0] = ax0[0];
    wire [6:0]
    sum1,sum2,sum3,sum4,sum5,sum6,sum7;
    wire [6:0] c1,c2,c3,c4,c5,c6,c7;
    wire [7:0] PP_7, PP7;
    wire cout_2s_compliment;

    my_xor gate1 ( PP_7[7], ax7[7],
    X[7]);
    my_xor gate2 ( PP_7[6], ax7[6],
    X[7]);
    my_xor gate3 ( PP_7[5], ax7[5],
    X[7]);
    my_xor gate4 ( PP_7[4], ax7[4],
    X[7]);
    my_xor gate5 ( PP_7[3], ax7[3],
    X[7]);
    my_xor gate6 ( PP_7[2], ax7[2],
    X[7]);
    my_xor gate7 ( PP_7[1], ax7[1],
    X[7]);
    my_xor gate8 ( PP_7[0], ax7[0],
    X[7]);
    RCA #(.N(8)) RCA_2S_compliment( PP_7
    , 8'b00000000 , X[7] , PP7 ,
    cout_2s_compliment);

    CSA_7bit
    CSA1(ax1[6:0],ax0[7:1],7'b0,sum1,c1);
    assign Pro[1] = sum1[0];
    CSA_7bit
    CSA2({ax1[7],sum1[6:1]},ax2[6:0],c1,sum2
    ,c2);
    assign Pro[2] = sum2[0];
    CSA_7bit
    CSA3({ax2[7],sum2[6:1]},ax3[6:0],c2,sum3
    ,c3);
    assign Pro[3] = sum3[0];
    CSA_7bit
    CSA4({ax3[7],sum3[6:1]},ax4[6:0],c3,sum4
    ,c4);
    assign Pro[4] = sum4[0];
    CSA_7bit
    CSA5({ax4[7],sum4[6:1]},ax5[6:0],c4,sum5
    ,c5);
    assign Pro[5] = sum5[0];
    CSA_7bit
    CSA6({ax5[7],sum5[6:1]},ax6[6:0],c5,sum6
    ,c6);
    assign Pro[6] = sum6[0];
    CSA_7bit
    CSA7({ax6[7],sum6[6:1]},PP7[6:0],c6,sum7
    ,c7);
    assign Pro[7] = sum7[0];

    wire [6:0] rca_vec;
    assign rca_vec = { PP7[7], sum7[6:1]
    };

```

```

    RCA #(.N(7)) RCA1(rca_vec, c7, 1'b0
    , Pro[14:8], Pro[15]);

```

endmodule

Results:

Input:

The input samples are the from sinusoid signal with low amplitude noise.

```

x = 0.25*sin(2*pi*50*t) +
0.1*sin(2*pi*1800*t);

```

```

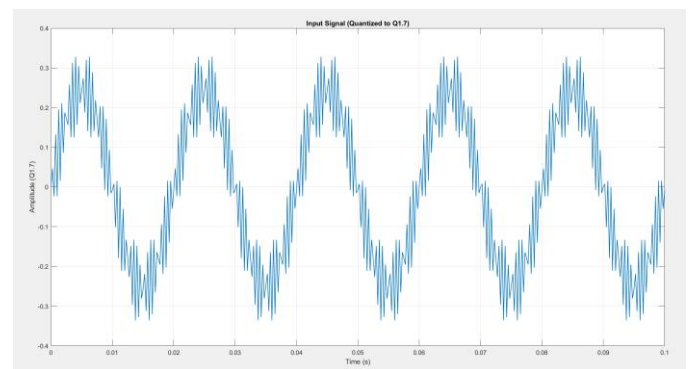
%% Fixed-point Q1.7 quantization
F = fimath('RoundingMethod','Floor');

```

```

xq = fi(x, 1, 8, 7, 'fimath', F); %
signed, 8-bit, 7 fractional bits

```



Output:

